

Part II

Evolution of Reinforcement Learning for LLMs

Author of Hands-On Large Language Models (book)

Our goal is to transition from human feedback to verifiable rewards. If we start with an untrained LLM, pre-training is language modeling to create a base LLM that can use tokens to predict the next token, e.g., given “The cat sat”, predict the next word. This pretrained model works well since it learns a bunch of linguistic and general information like what certain words mean in different contexts, e.g., the word “bank” as a river versus a financial institution. The next step is supervised fine-tuning the LLM using existing subject matter. Our input is a prompt/query and our outputs are answers for ChatGPT-like behavior. Unfortunately, this is very sensitive to over-fitting, which has less opportunity for other potential, correct answers. One way to fine-tune the pretrained model is via **preference fine-tuning**, in which you have a prompt/query with multiple answers, but rewards answers based on preference instead of just correctness, e.g., safer or longer. This requires a lot of data to score. One example of preference fine-tuning is Reinforcement learning for Human Feedback (RLHF) via the PPO algorithm. After that, there is a **reward fine-tuning** step where one creates verifiable feedback and assign rewards. Verifiable in this context means there is a factual answer. For example, determine whether or not the code generated by an LLM compiles. An example of reward fine-tuning is Reinforced learning with Verifiable Reward (RLVR) via the PPO/GRPO algorithms.

Definition 3.1. POLICY - ??? a rule used by the LLM to decide which actions to take

Definition 3.2. POLICY MODEL - ??? the LLM you want to fine tune

Definition 3.3. Frozen model – A model who’s weights never change, i.e., only used for inference.

Definition 3.4. Reference model – An exact copy of the policy model at the start of training, and is frozen.

Definition 3.5. Value model – estimates how good a certain state is. estimates how good he output will be based on what we have thus far, e.g., multi-turn conversations or input to output tokens.

Definition 3.6. Reward model – the model that assigns values to the value model’s outputs

The policy generates answers that are assigned a reward model to generate a value, e.g., pass/fail that we can use to update the policy. The reward model is frozen so it does not get updated during training in order to protect itself from overfitting to the policy model.

Start with a **policy model**, i.e., a model we want to fine tune. Let the input be a query, and the output be an answer. The **value model** calculates values for the output’s individual portions, whereas the **reward model** judges the output as a whole via a single value. In RLHF, the reward model was the labeled preference data, because the scores assigned to preferred outputs are fixed. In RLVR, the reward model can be a simple python function that checks the output’s formatting or length.

4 KL-Divergence

Kullback-Leibler (KL) divergence is a function that maximizes rewards by checking the distribution of output tokens. Given two probability distributions, P and Q ,

$$D_{KL}(P \parallel Q) = \sum_{x \in X} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

Observe this is not symmetric, and does not satisfy the triangle inequality. However, $0 \leq D_{KL}(P \parallel Q)$ since log is concave.

When training LLMs, P and Q are token probability distributions, which are typically the softmax of the logits layer, i.e., for a vocabulary word i , token x_i (token at index i) with corresponding logit on the LM head z_i ,

$$P(x_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

For training reasoning models like *R1* where we want to train the model to get new behaviors like reasoning, we can train the model for a number of steps, and then after say, 50 steps, that new checkpoint becomes our new reference model for the following 50 steps. And we keep cycling them like this.

5 Proximal Policy Optimization

Start with a policy model. Let the input be a query, and the output be an answer. The output is passed to the reference, reward, and value models. The reference & reward models together assign the output a single reward value. We use the reference model to prevent the policy model’s token distribution from drifting or overfitting via **KL divergence**. In practice, this means we use a frozen copy of the policy model with its token output layer removed to only get the logits layer. The entire generated sequence from the policy model goes into the reference model to get its probability distributions for the tokens, and we want that distribution to be similar between the reference and policy models. The **value model** creates a value that grades the output e.g., “I think this sentences gets a 7/10,” and its goal is to determine how good the generation is so far on average. The value model is typically an LLM that needs to generate a scalar score instead of generating tokens. It is usually trained during reinforcement learning to make sure it aligns with the policy model. Typically, this means take the policy model and replace the token output layer with a classification layer that can calculate a score.

Definition 5.1. Advantage

We use the output reward & value to calculate the **advantage** via comparison. If the reward is better than the value, then we have a positive advantage. For example, a reward of 8 versus a value of 7 could calculate the advantage as $8 - 7 = +1$. The advantage going up implies the policy gets updated with positive feedback weighted by the advantage, and the value model also needs to be updated to reflect the new scoring. But what happens when the advantage is negative? In practice, only the highest advantage scenarios affect the model.

5.1 Group relative policy optimization

PPO was too complicated because the value model is unstable because it can drift from the policy model and is expensive to train, so they aimed to remove it. Given policy model π , input query q , output multiple answers o_i , the outputs are passed to the reference model and reward model separately. The reward model calculates the reward r_i for each output o_i , and we calculate a group computation f , then perform a round-robin to calculate the advantages, i.e.,

$$Adv = \left\{ f \left(\{r_i\}_{i=1}^N \right) - r_i \right\}_{i=1}^N$$

So the rewards get aggregated (often as an average), and then we calculate the difference between each reward and this aggregation to get the **advantage**. These advantages are compared against the reference model to prevent drifting via KL-divergence, and are also used to update the policy model. The scores of the group are normalized to improve the learning signal.

6 Applications

deepseek-r1. tool integrated reasoning. multi-modal reinforcement learning. test-time reinforcement learning.

deepseek-r1 combines preference and reward together. 1. supervised fine tuning with 800K high quality reasoning&non-reasoning samples. 2. use GRPO to with input question, output multiple answers. Verifiable criteria is formatting rewards (cheap), & accuracy rewards (cheap), preference rewards (costly, usually use llms). compiling is not a matter of preference, but preference is RLHF. Convergence is based on weights or bias.

tool-integrated reasoning. how can an llm be trained to call tools before reasoning. input is query, model reasons and outputs structure to pass through a tool, the tool gives back the output to the llm. tool-integrating just like using a calculator in the backend to verify the answer. toolRL paper reference. input is query, model is reasoning, formatting rewards and tool rewards to verify correct tool name and correct parameters. call GRPO to update model based on rewards. Doesn't check if correct name is being called instead of the correct tool.

multi-modal RL, how can reinforcement learning help multi-model llms reason about inputs? input is data, e.g., picture with question, reasoning, answer information. reinforcement calculates formatting rewards (thinking and answer tokens are present) and accuracy rewards (number of objects in an image). **constructing rewards is the most difficult part because they need to make sense of the use case.**

test-time RL. what if you have no labeled data and still want to do reinforcement learning? input query, output multiple answers. choose the most frequent answer and reward getting the most frequent answer. reward gets passed to GRPO. crazy since $5+3$ could produce 7 as the most frequent answer, but it will be rewarded. However, the other ones will still be considered infrequent.

7 References

- <https://arxiv.org/abs/1707.06347> - PPO paper
- <https://arxiv.org/abs/2402.03300> - GRPO paper
- <https://arxiv.org/abs/2504.16084> - Test-Time RL paper